

Prediction Markets Dashboard

Cross-Exchange Analytics for Polymarket × Kalshi

Nikhil Pothuganti · Technical Submission · July 20, 2026

Next.js 15 · TypeScript · React 19 · 6,372 LOC across 57 modules · 3 routes · 11 API endpoints

1 Overview

Prediction markets are fragmented. The same real-world question — who wins the 2028 US presidential election, who lifts the 2026 World Cup — trades simultaneously on Polymarket and Kalshi, at different prices, under different resolution rules, in different units, behind different APIs. A price gap between the two venues looks like free money.

It usually is not. This dashboard is built around that claim. It ingests live order books and price history from both venues, normalises them into one model, infers which markets on either venue are *the same question*, and then subjects every apparent gap to three tests a naive spread scanner skips: **is it fillable** (walk both books, don't quote the touch), **does it survive fees and time** (non-linear Kalshi taker fee; annualise over capital lockup), and **do the two markets actually pay the same** (resolution-equivalence assessment).

Design thesis. A visible price gap is by default **not** an edge. A spread is presented as tradeable only once the match is `VERIFIED`, the depth- and fee-aware net edge is positive, and the divergence assessment is shown beside it.

Headline result. On the 2028 President market, an apparent 8-point gap resolves to roughly **0.8%/year** once you cross the spread instead of quoting mids, pay the Kalshi fee, and account for capital locked until January 2029. A T-bill beats it. Separately, the two venues do not settle on the same event — Kalshi pays on who is *inaugurated*, Polymarket on the *AP/Fox/NBC race call* — so the position is not a lock at all.

Every design decision follows from that thesis: it is why the matcher has four guard rails and rejects rather than guesses, why the arbitrage engine walks the book, why returns are annualised, and why the divergence layer exists. The system is built to talk you out of bad trades as readily as it surfaces good ones.

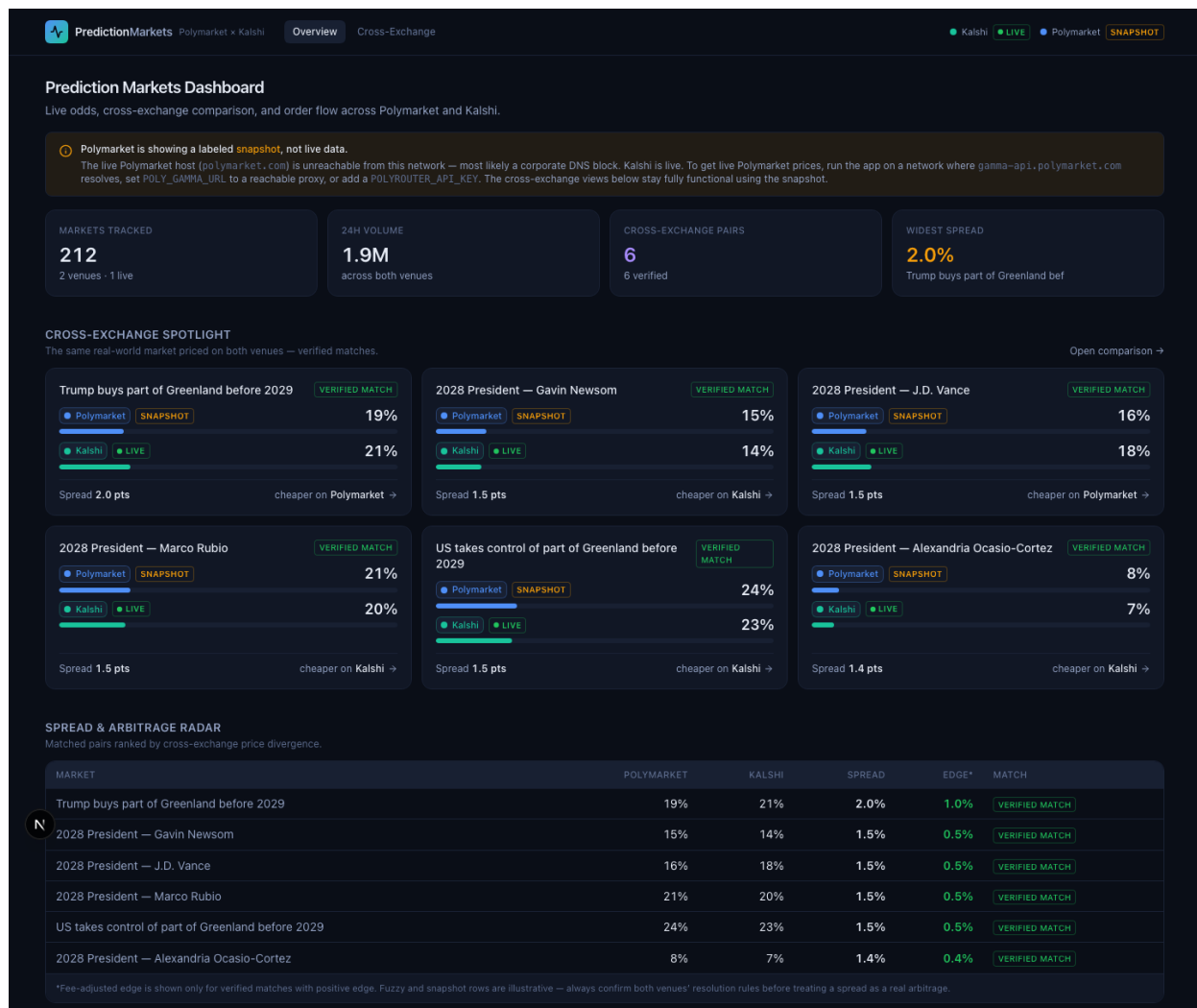


Figure 1. Overview page. Per-venue badges (top right) show **Kalshi** LIVE and **Polymarket** in labeled **SNAPSHOT** mode — the degradation path of §4.6. Below: corpus statistics, verified matched pairs, and the arbitrage radar ranked by divergence. The fee-adjusted-edge caveat is stated inline under the radar rather than hidden.

2 Architecture

A single Next.js 15 App Router deployment. There is no separate backend: the server tier is a set of route handlers under `app/api/`, and they are the only component that talks to an upstream venue. That indirection is not incidental — **Kalshi** returns 403 to any request carrying an `Origin` header, so a client-side fetch cannot work at all. Proxying server-side also keeps optional keys off the client and gives one place to normalise, cache and degrade.

The weight of the codebase sits in `lib/` (3,297 LOC across 21 modules) rather than `components/` (2,342 across 19) — this is a data pipeline with a UI, not a UI with a data source.

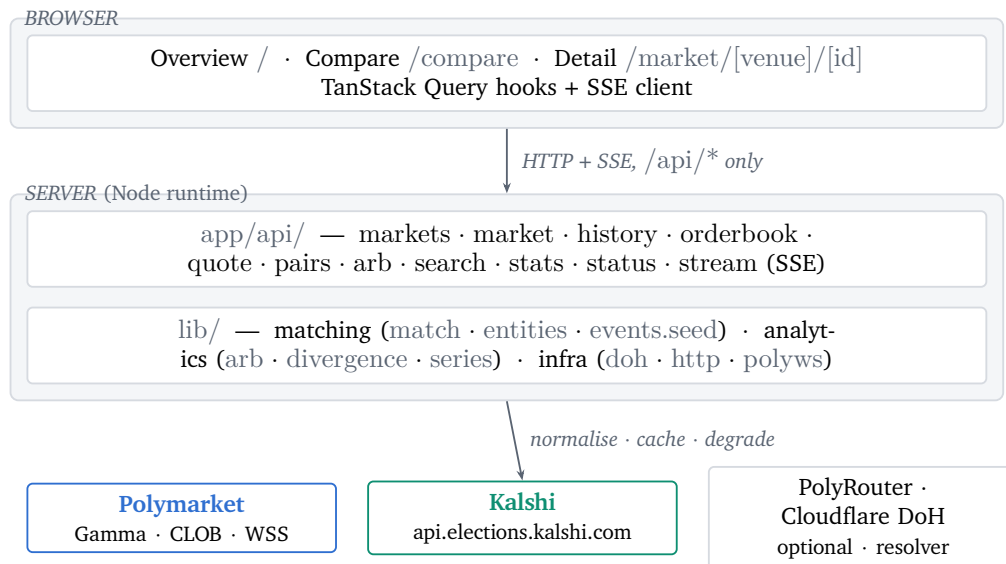


Figure 2. Request topology. The browser talks only to `/api/*`; every upstream call is made server-side. Corpus and pair caches are prewarmed at boot (`instrumentation.ts`) so the first user request does not pay the ~17 s cold build.

3 Features

Feature	What it does	§
Cross-exchange comparison	Dual-line probability history for the same question on both venues (1D/1W/1M/3M/All), live spread, fee-adjusted edge, both order books, resolution rules side by side	4.3
Depth-aware arbitrage	Walks both books level-by-level until the edge collapses; non-linear Kalshi fee; reports fillable size, net profit, ROI, and ROI annualised by resolution date	4.3
Resolution-divergence layer	Per-event assessment of whether two matched markets actually settle identically, with concrete divergence cases and a worst-case narrative	4.4
Three-tier matching	Event-level entity alignment, curated seed, then IDF-weighted fuzzy fall-back behind four guard rails	4.2
Real-time order book	Streams Polymarket 's book tick-by-tick over a server-side WebSocket bridged to the browser via SSE; Kalshi polls at 2 s (its socket needs auth)	4.5
Arbitrage radar	Matched pairs ranked by divergence with category filter and sort; net edge shown only for verified matches with positive depth-aware edge	4.3
Market browser + search	Thousands of markets, server-side sorted and filtered, debounced search fanning out to both venues, infinite scroll	—
DNS-over-HTTPS reachability	Resolves Polymarket over DoH and connects by IP with correct SNI, so live data works on networks that DNS-block the domain — no VPN, no proxy	4.6
Per-venue degradation	Each venue reports LIVE/SNAPSHOT/OFFLINE; the UI stays functional if one source is unreachable, with the mode labeled everywhere	4.6
Performance engineering	Five cache layers with stale-while-revalidate and in-flight dedup; boot-time prewarm; incremental chart updates that preserve zoom/pan	4.7

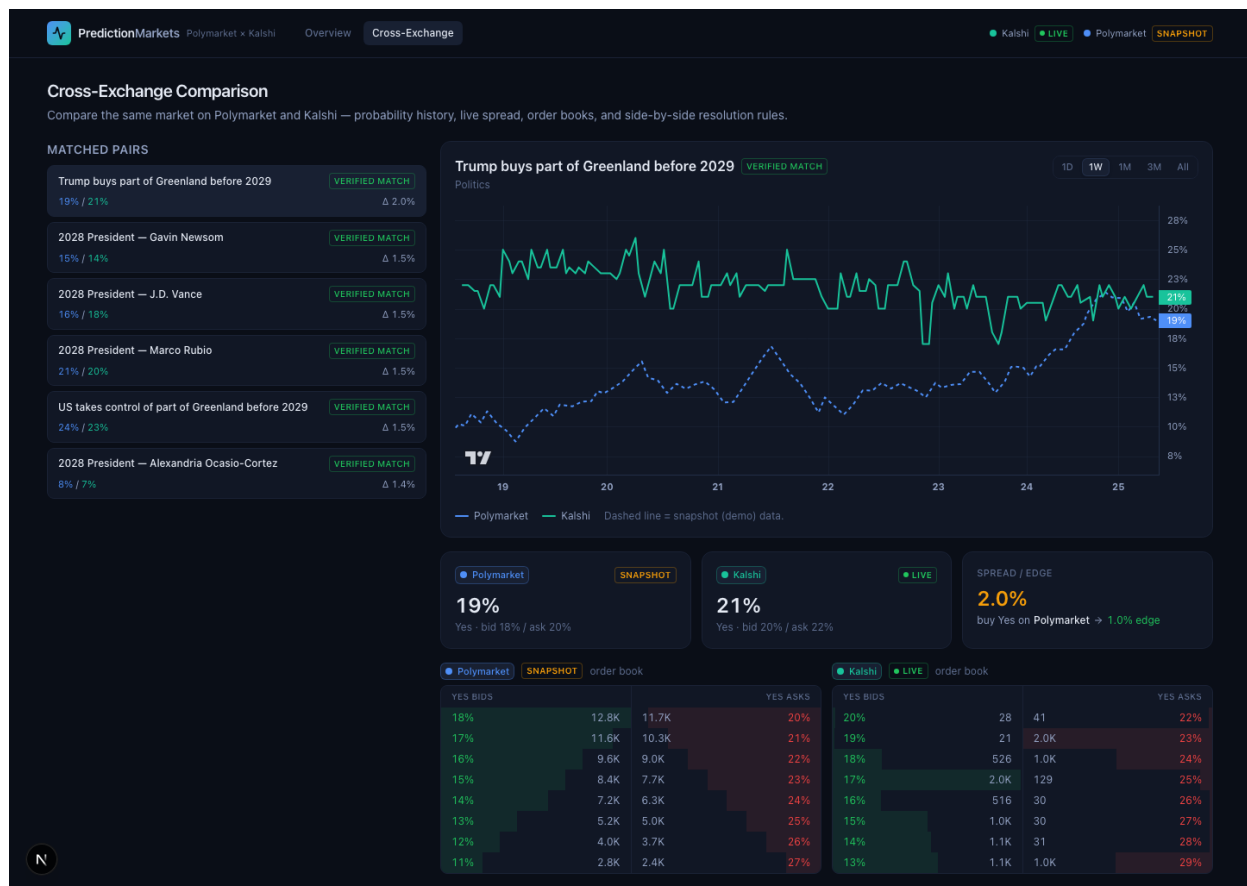


Figure 3. Cross-exchange comparison. Left: matched pairs with per-venue prices and Δ . Right: dual-line probability history (dashed = snapshot, so provenance is visible in the mark itself), price and spread tiles, and both venues’ full-depth books. The spread tile reports the raw 2.0% gap *and* the 1.0% fee-adjusted edge (§4.3). The panel recomputes arbitrage client-side from the streamed books, so the number tracks the book on screen.

4 Core Technical Work

4.1 Normalisation

Everything upstream is coerced into one model (`lib/types.ts`) under a single invariant: **all prices and probabilities are floats in $[0, 1]$** — never cents, never 0–100. Conversion happens once, inside each venue adapter. A Market carries a namespaced id, an outcome array, optional `tokenIds` parallel to the outcomes (**Polymarket**, for CLOB calls), optional `seriesTicker/eventTicker` (**Kalshi**, for candlestick URLs), and the venue’s resolution rules text, which the divergence layer consumes. Upstream wire shapes stay inside their adapter as private interfaces.

One correctness guard is worth noting: deriving a price from a two-sided quote is usually the midpoint, but a market maker parking a $1\phi/99\phi$ quote yields a meaningless “50%”. Any two-sided quote wider than 90ϕ is therefore treated as a placeholder, deferring to last trade or reporting null rather than a confidently wrong 50%.

4.2 Cross-exchange matching

The hardest problem here. With no shared identifier the join must be inferred from text, and a false positive is worse than a miss — it manufactures a phantom spread between two questions that are not the same. Three tiers run in sequence, each consuming markets from a shared pool so later tiers see only what earlier ones could not claim.

Tier		Method	Mechanism	Confidence
0	—	Entity alignment	A registry maps a Polymarket event slug to a Kalshi series, then aligns <i>every</i> outcome by canonical entity. 6 events, ~108 sports pairs.	VERIFIED
1	— Seed	Curated	6 hand-verified pairs, each carrying both a live Polymarket conditionId and a snapshot id so they populate in either data mode.	VERIFIED
2	—	IDF-weighted overlap	Tokenise with stopwords, stemming and alias expansion; weight by inverse document frequency; block via an inverted index; threshold at 0.6.	FUZZY

Table 1. The matching pipeline. Confidence degrades down the stack, and only VERIFIED pairs are ever eligible to be presented as arbitrage.

Tier 0 — event-level alignment

A text matcher applied to a categorical event systematically under-performs: it only “sees” the high-volume candidates whose titles happen to be similar. Instead a registry ([lib/events.seed.ts](#)) declares the correspondence at the *event* level — a [Polymarket](#) event slug and a [Kalshi](#) series ticker — and outcomes are then aligned by canonical entity, so the whole field pairs. Six events are registered: 2028 President, the two 2028 nominations, the 2026 World Cup (~46 countries), the 2027 NBA champion (30) and the 2027 Stanley Cup (32). Because the [Kalshi](#) series is fetched directly rather than read from the paginated corpus, a series below the top-volume pages still contributes its full field. Adding a league is one registry entry.

Canonicalisation ([lib/entities.ts](#)) keeps alias tables for persons and countries — including a deliberate carve-out keeping *Donald Trump Jr* distinct from *Donald Trump*, exactly the collision that would otherwise produce a large spurious spread. Teams use token overlap instead, so a new league needs no roster data: Sacramento matches Sacramento Kings, and LA Clippers separates from LA Lakers on the distinctive token.

Tier 2 — IDF-weighted fuzzy matching

Let $T(m)$ be the token set of market m after stopword removal, stemming and alias expansion. Over a pool of N documents from both venues:

$$\text{idf}(t) = \log\left(1 + \frac{N}{\text{df}(t)}\right), \quad W(m) = \sum_{t \in T(m)} \text{idf}(t), \quad S(p, k) = \frac{\sum_{t \in T(p) \cap T(k)} \text{idf}(t)}{\min(W(p), W(k))}. \quad (1)$$

A pair is accepted only if $S \geq 0.6$ subject to two admissibility conditions:

$$|T(p) \cap T(k)| \geq 2 \quad \text{and} \quad \exists t \in T(p) \cap T(k) : \text{df}(t) \leq 8. \quad (2)$$

The second — requiring a *distinctive* shared token — is what makes the tier usable. Every 2028 candidate market shares 2028, president and nominee; without a rarity requirement they all match each other at high scores. Demanding a token appearing in at most 8 documents forces the match to rest on a name. Normalising by $\min(W(p), W(k))$ rather than the union is deliberate: venues write titles at different verbosity, and a terse [Kalshi](#) title should still match a wordy [Polymarket](#) one that contains it.

Cost is bounded by an inverted index (only [Kalshi](#) markets sharing ≥ 2 tokens are scored, capped at 40 candidates) and by considering only the 600 most liquid [Polymarket](#) markets.

Guard rails

Four guards reject a candidate before scoring; two were added specifically to kill bad matches found in testing.

- **Question kind** — each market is tagged *nominee* / *run* / *winner* / *host* and cross-kind matches are rejected. This keeps “will country X *win* the World Cup” apart from “will country X *host* it”: same entity, near-identical tokens, wildly different prices.
- **Multi-entity** — a combined “A, B or C” market never matches a single-entity one, however many name tokens they share.
- **Numbers** — conflicting numerals or years separate a “ $\geq 25\text{bps}$ ” market from a “50bps” one.
- **Category** — categories must be equal; cheap and high-precision.

Post-hoc, a fuzzy pair with spread > 0.35 is dropped: two genuinely equivalent liquid markets do not disagree by 35 points, so such a gap is far more likely a title collision.

4.3 Depth-aware arbitrage

The naive computation is systematically wrong in the optimistic direction. Differencing mid-prices gives an edge you cannot transact — you buy at the ask, sell at the bid, and the size at the touch is often trivial.

On a binary market, holding YES on one venue and NO on the other redeems for exactly \$1 regardless of outcome. Buying NO at $1 - b$ is equivalent to selling YES at bid b , so for a share bought at ask a and hedged against bid b :

$$\text{net edge/share} = b - a - \phi_{\text{buy}}(a) - \phi_{\text{sell}}(1 - b), \quad \text{capital/share} = a + (1 - b), \quad (3)$$

with $\phi_{\text{kalshi}}(p) = 0.07 p (1 - p)$ and $\phi_{\text{polymarket}}(p) = 0$. The **Kalshi** fee is maximised at $p = 0.5$ and vanishes at the extremes, so a flat fee assumption misprices exactly the mid-probability markets where spreads are widest.

The engine performs a two-pointer greedy match — buy-venue asks ascending against sell-venue bids descending, consuming $\min(\text{remaining ask}, \text{remaining bid})$ at each step and halting the moment the fee-adjusted edge goes non-positive. Both directions are evaluated and the more profitable returned.

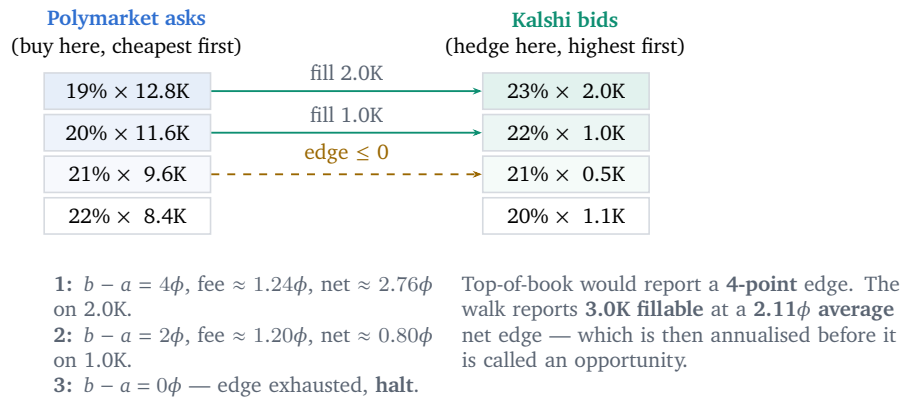


Figure 4. The book walk (illustrative sizes). The reported edge is an average over filled size rather than the touch; `bestEdge` retains the touch for reference, `avgEdge` is what the position earns.

Annualisation — the decisive step

A cross-exchange lock is not a free roll: capital $a + (1 - b)$ per share is committed until resolution, which for the 2028 President market is over two years out. So

$$\text{ROI} = \frac{\text{net profit}}{\text{capital}}, \quad \text{ROI}_{\text{ann}} = \frac{\text{ROI}}{\tau}, \quad \tau = \frac{t_{\text{close}} - t_{\text{now}}}{365.25 \text{ days}}, \quad (4)$$

reported only when $\tau > 0.02$ years (~ 7 days), below which annualising produces meaningless multipliers. The result is benchmarked against a configurable risk-free rate (default 6%/yr): the UI colours ROI_{ann} green when it clears the benchmark, amber when a Treasury bill would have done better. This is the single most important number in the product — it is what turns the 8-point gap of \$1 into 0.8%/year, shown in amber.

Two spread notions, deliberately

`lib/match.ts` computes a naive $|p_{\text{poly}} - p_{\text{kalshi}}|$ with a flat 1% fee buffer: a *ranking pre-filter* that runs over every pair on every corpus rebuild, must be cheap, and needs no order book. `lib/arb.ts` does the full depth- and fee-aware walk: the *decision number*, computed only for pairs that warrant it. The radar therefore ranks by the naive spread while reporting the rigorous figure — net profit scales with size deployed and so is a poor sort key, whereas raw divergence is what you want to scan by. §5 notes the maintenance cost.

4.4 Resolution divergence

Price analysis cannot detect the failure that matters most: two markets that are the same question by every textual and statistical measure but *settle differently*.

Kalshi resolves the 2028 presidential market on who is **inaugurated** on 20 January 2029. **Poly-market** resolves on who **wins the election** as called by AP, Fox and NBC, days after 7 November 2028. In the clean case they agree. They differ in source of truth, in threshold (winning versus taking office), and in timing — and that gap is where an arbitrage “lock” stops being a lock.

Worst case, as surfaced in the UI. Hold YES-**Polymarket** + NO-**Kalshi** on candidate X . The networks call X in November 2028, so **Polymarket** resolves YES — that leg loses. X then dies or is replaced before inauguration, so someone else is inaugurated and **Kalshi** resolves NO for X — *the hedge leg loses too*. A position a naive scanner markets as risk-free loses on both sides.

An LLM equivalence judge was run over each matched event’s actual resolution text and its output reviewed, classifying each equivalent / minor / divergent with a confidence, concrete divergence cases, and a worst-case narrative. Results are baked into `lib/divergence.ts` rather than evaluated per request — resolution criteria change on the order of years, so an LLM call per request would be latency for no freshness. Unjudged pairs fall back to a generic politics warning rather than silently appearing safe, and the assessment surfaces three ways: a panel, a per-pair flag, and a red warning inside the arbitrage panel.

The judge is not merely rubber-stamping: both nomination markets came back *minor* rather than equivalent, because **Polymarket** carries an explicit clause locking the result against a post-convention replacement while **Kalshi** is silent — immaterial in almost every world, but not identical.

4.5 Real-time data path

Polymarket’s CLOB publishes book and `price_change` over WebSocket. The browser cannot consume it directly (same CORS and DNS constraints as the REST path), so the server holds the socket and bridges it over Server-Sent Events, with a 15 s heartbeat and unsubscribe wired to the request’s abort signal. The client hook layers SSE over a 2 s poll: the poll paints instantly on mount, the stream takes over once connected, avoiding the blank-then-populate flash of a stream-only implementation. **Kalshi** requires API-key auth and request signing for its socket (REST market data is public, the socket is not), so its book uses a 2 s poll — a documented consequence of the venue’s auth model, not an implementation gap.

4.6 Reachability: DNS-over-HTTPS

Polymarket is DNS-blocked on some networks — notably in India, where ISP resolvers refuse *.polymarket.com. The block is *DNS-only*: the servers and their TLS endpoints remain reachable given an address. The app therefore resolves **Polymarket** over DoH (Cloudflare by default) and hands `undici` a custom lookup so `fetch` connects to the resolved IP while presenting the correct SNI and Host. The same technique is applied to the WebSocket. Live **Polymarket** data works with no VPN and no proxy.

The full chain is **direct DNS** → **DoH** → **PolyRouter** (only if `POLYROUTER_API_KEY` is set) → **stale cache** → **labeled snapshot**. The snapshot is not a static fixture — it generates deterministic synthetic history from a seeded PRNG and a synthetic $\pm 1\phi$ book, so charts and ladders stay functional and reproducible. Crucially `mode: "snapshot"` propagates through the type system to every badge and renders history dashed, so the UI can never present demo data as live (visible in Figure 1, where the banner names the block and states the remedy).

4.7 Performance and freshness

The corpus and pair build costs ~17 s cold. Three mechanisms keep that off the critical path. **Stale-while-revalidate caching** — corpus (180 s), pairs (15 s) and arbitrage (20 s) caches serve stale instantly on expiry and rebuild in background, with in-flight dedup so a thundering herd collapses to one fetch; only the very first build ever blocks. **Boot-time prewarming** — an instrumentation hook drives pairs and arb every 5 s for six attempts, then every 150 s, deliberately inside the 180 s TTL. **Incremental rendering** — charts update through the library's `update()` path and the book streams over SSE, so only the moving point repaints and the user's zoom/pan is never reset by a refresh.

One subtlety shows the degradation philosophy: if **Polymarket** is live but the event fetch yielded no event-level pairs, the cache entry is *backdated* to expire in ~2 s rather than serving a thin result for the full TTL. A transient failure degrades for seconds, not a quarter-minute.

4.8 Time-series alignment

The charting library positions points by *index*, not timestamp. Feeding it irregular points produces a time axis whose labels do not match horizontal positions and, worse, two venues' series that drift out of alignment with each other. `lib/series.ts` resamples every series onto a shared uniform grid per range (capped at 2,200 buckets) with forward-fill, so index is proportional to time and the two venues line up exactly at every x . Without this the dual-line comparison in Figure 3 would be quietly misleading.

5 Limitations and Future Work

Stated plainly, because a submission that hides its gaps is less useful than one that maps them.

5.1 Known gaps

No automated tests.

The most conspicuous gap. The highest-value targets are pure and dependency-free: the fuzzy scorer and its guards, the arbitrage book walk, and the series resampler. The walk especially — a two-pointer accumulator decrementing size across two ladders is exactly where an off-by-one is silent and yields a plausible-but-wrong number. First thing I would add.

Fee model in three places.

The flat 1% buffer appears in the matcher and again in the comparison view, while the accurate non-linear curve lives in the arbitrage engine. The split is deliberate (§4.3), but the constant should be defined once so a fee change cannot leave the paths disagreeing.

Cache pattern duplicated.

The SWR plus in-flight-dedup idiom is written four times with slightly divergent error handling; a generic `swrCache<T>(ttl, builder)` would collapse all four. DoH resolution is likewise implemented

twice (HTTP and WebSocket) with independent caches.

Dead components.

`SpreadTable.tsx` and `Sparkline.tsx` are unreferenced (`SpreadTable` was superseded by the radar), and the README still describes a “spread table” on the overview page. Both should go and the line be corrected.

No CI, container or deploy manifest.

Development-grade setup only.

5.2 Inherent constraints

- **One-minute granularity floor** — both venues cap history there. Sub-minute movement is observable only through the live book and quote.
- **Volume units are not comparable** — [Polymarket](#) reports USD, [Kalshi](#) contract count. Labeled per card and deliberately not converted, since any conversion would be a fabrication.
- **Divergence coverage is partial** — three events individually judged, a conservative generic warning elsewhere.
- **Execution is out of scope** — the system identifies and sizes opportunities; it does not trade, and makes no claim about slippage between observation and fill.

5.3 Next steps, in priority order

1. Unit tests for the matcher, book walk and resampler; property tests asserting the walk never reports negative net edge or size exceeding available depth.
2. Extract `swrCache<T>` and a single fee module; delete the dead components.
3. Extend the event registry (more leagues, non-US elections) and re-run the divergence judge across it.
4. Persist a spread time series so the radar can rank by *unusual* divergence relative to a pair’s own history rather than absolute gap.
5. Add [Kalshi](#) WebSocket support behind API-key signing for real-time parity.

The most valuable output of this system is frequently negative — an amber annualised ROI under the risk-free line, or a divergence flag explaining how a “lock” loses both legs. A tool that only ever finds opportunities is not measuring anything.

Appendix

Self-contained reference: API surface, data model, configuration, algorithms and operational behaviour, sufficient to run, extend or review the system without reading the source.

A Repository Layout

```
instrumentation.ts  Boot hook: prewarm + keep-warm the corpus/pairs caches
app/api/           11 route handlers -- the entire backend
app/page.tsx       Overview: stats, spotlight, arb radar, market grid
app/compare/       Cross-exchange comparison
app/market/[venue]/[id]/  Single-market detail
lib/ (21 modules, 3,297 LOC)
types.ts           Normalized Market / Orderbook / MarketPair / Divergence / ArbResult
kalshi.ts          Kalshi provider: events, candlesticks, orderbook (SWR corpus)
polymarket.ts      Polymarket provider: Gamma/CLOB -> PolyRouter -> snapshot
polyws.ts          Polymarket order-book WebSocket (over DoH) -> SSE
polyrouter.ts      Optional aggregator fallback
snapshot.ts        Labeled demo dataset + seeded synthetic history
doh.ts             DNS-over-HTTPS resolver for DNS-blocked upstreams
aggregate.ts       Corpus -> cross-exchange pairs (cached, SWR)
match.ts           Three-tier matcher: event / seed / IDF-weighted fuzzy
entities.ts        Person / country / team canonicalizer + alias tables
events.seed.ts     Event registry (Polymarket event <-> Kalshi series)
pairs.seed.ts      Curated hand-verified pairs
arb.ts             Depth-aware fillable arbitrage
divergence.ts      Resolution-equivalence assessments + lookup
series.ts          Resample irregular points onto a uniform time grid
http.ts            Fetch with timeout + backoff; never sends Origin
config.ts          Centralized env-overridable configuration
hooks.ts / api.ts  TanStack Query hooks (incl. SSE) + typed client
components/ (19 components, 2,342 LOC)
CompareView        ProbabilityChart ArbRadar ArbPanel DivergencePanel MarketGrid
MarketCard MarketDetail OrderbookDepth DepthChart PairsSpotlight PairCard
PairFilterBar StatsRow StatusBanner Header ui
```

B HTTP API

All endpoints are GET-only, Node runtime, force-dynamic, no authentication, no mutating endpoints. stale-while-revalidate is set to 6× s-maxage.

Endpoint	Parameters	Returns	s-maxage
/api/markets	q, category, venue, sort, off-set, limit	PagedMarketsResponse	30 s
/api/market/[venue]/[id]	path params	Market	15 s
/api/orderbook/[venue]/[id]	path params	Orderbook	1 s
/api/stream/orderbook/[venue]/[id]	path params; 400 for kalshi	text/event-stream	—
/api/history	range, repeated m	HistoryResponse	60 s
/api/quote	repeated m (max 12)	QuoteResponse	1 s
/api/pairs	—	PairsResponse	30 s
/api/arb	—	ArbResponse	10 s
/api/search	q	SearchResponse	60 s
/api/stats	—	StatsResponse	30 s
/api/status	—	VenueStatus[]	15 s

Table 2. /api/markets with q also fans out to [Polymarket](#)’s public search, so any market is reachable even outside the prefetched corpus. /api/quote memoises 2s in process. The SSE stream emits data: frames carrying a serialised Orderbook plus a 15 s comment heartbeat, torn down on the request’s abort signal.

C Data Model

Invariant: every price and probability is a float in $[0, 1]$ — not cents, not 0–100. Conversion happens once, inside each venue adapter. Upstream wire shapes are private to their adapter and never cross into the domain model.

Type	Definition
Venue	"polymarket" "kalshi"
DataMode	"live" "snapshot" "unreachable" — drives every UI badge and the dashed-line chart treatment
Category	Politics, Economics, Crypto, World, Tech, Culture, Sports, Other
Outcome	id, name, price, bid, ask
Market	id = $\{\text{venue}\}:\{\text{nativeId}\}$; title, subtitle, category, venue, mode; yesPrice, bid, ask; volumeTotal, volume24h; closeTime; rules (resolution text, read by the divergence layer); tokenIds parallel to outcomes (Polymarket); seriesTicker/eventTicker (Kalshi)
PricePoint	$\{ t, p \}$ — t in unix seconds, p in $[0, 1]$
Orderbook	bids descending, asks ascending, of $\{ \text{price, size} \}$
VenueStatus	venue, mode, ok, latencyMs, message
MarketPair	pairId, title, category, polymarket, kalshi, confidence (verified fuzzy unmatched), matchMethod (manual fuzzy sports none), spread, edge, cheaperVenue, notes
Divergence	status (equivalent minor divergent), confidence, divergenceCases[], reasoning, worstCase
ArbResult	hasArb, buyVenue, sellVenue, fillableSize, grossProfit, netProfit, capital, roi, annualizedRoi, avgEdge, bestEdge

D Upstream Endpoints, Caching and Configuration

Venue	Endpoint	Purpose
Kalshi	/events?with__nested__markets	Corpus, paginated (200 events/page)
Kalshi	/markets/{ticker}[/orderbook]	Single market; full-depth ladder
Kalshi	/series/{s}/markets/{t}/candlesticks	Probability history
Polymarket	Gamma /markets, /events?slug=	Corpus (100/page); event-level fetch
Polymarket	CLOB /book, /prices-history	Ladder; probability history
Polymarket	wss://ws-subscriptions-clob	Live book: book + price_change
Other	PolyRouter; Cloudflare /dns-query	Optional fallback (needs key); DoH

Table 4. Both venues’ read endpoints are public and need no key. **Kalshi** returns fixed-point cents and volume as contract count; **Polymarket**’s Gamma returns outcomes, outcomePrices and clobTokenIds as JSON-encoded *strings* and volume in USD. All outbound requests omit Origin, since **Kalshi** 403s any request carrying one.

Cache layer	TTL	Behaviour
Per-venue corpus	180 s	SWR; in-flight dedup
Cross-exchange pairs	15 s	SWR + dedup; backdated to ~2 s if Polymarket is live but produced no event pairs
Arbitrage rows	20 s	SWR
Quote memo / Polymarket events	2 s / 90 s	In-process
DoH / WebSocket IP	60–900 s / 300 s	Derived from record TTL

Table 5. Server caches. Client polling (TanStack Query, staleTime 30 s): status 20 s, stats 30 s, markets 60 s, pairs 20 s, arb 12 s, history 30 s, quote 3 s, book 2 s (SSE supersedes the book poll once connected). instrumentation.ts prewarms /api/pairs and /api/arb every 5 s for 6 attempts then every 150 s — inside the 180 s corpus TTL. It issues HTTP requests rather than importing lib/aggregate because importing pulls the upstream module graph into the instrumentation bundle, where Node built-ins fail to resolve.

Variable	Default	Effect
POLY_DOH	auto	auto = direct DNS then DoH on failure; on = always DoH; off = never
DOH_RESOLVER	Cloudflare	DoH endpoint
POLYROUTER_API_KEY	unset	Enables the PolyRouter fallback stage
KALSHI_BASE_URL	Kalshi v2	Override venue base
POLY_GAMMA_URL / POLY_CLOB_URL	Polymarket hosts	Override (e.g. a reachable mirror)
POLY_SNAPSHOT	on	off disables the demo fallback; UI shows OFFLINE
KALSHI_PAGES / POLY_PAGES	4 / 4	Corpus breadth vs cold-start latency
UPSTREAM_TIMEOUT_MS	12000	Per-request upstream timeout
NEXT_PUBLIC_RISK_FREE_RATE	0.06	Benchmark for annualised ROI; the amber/green threshold

Table 6. Every variable is optional — both venues’ public read endpoints work with no key. Copy .env.example to .env.local to override.

E Algorithm Pseudocode

```

WALK(buyAsks, buyVenue, sellBids, sellVenue):
  asks <- buyAsks sorted ASCENDING ; bids <- sellBids sorted DESCENDING
  i, j <- 0, 0 ; remainingAsk <- asks[0].size ; remainingBid <- bids[0].size
  size, gross, net, capital, bestEdge <- 0

  WHILE i < |asks| AND j < |bids|:
    a <- asks[i].price ; b <- bids[j].price
    IF b <= a: BREAK // no gross edge left
    fee <- FEE(buyVenue, a) + FEE(sellVenue, 1 - b)
    netPer <- b - a - fee
    IF netPer <= 0: BREAK // fees consumed the edge
    qty <- MIN(remainingAsk, remainingBid)
    IF qty <= 0: BREAK

    IF size == 0: bestEdge <- netPer // edge at the touch
    size += qty ; gross += (b - a) * qty ; net += netPer * qty
    capital += (a + (1 - b)) * qty // YES leg + NO leg
    remainingAsk -= qty ; remainingBid -= qty
    IF remainingAsk == 0: remainingAsk <- asks[++i].size

```

```

    IF remainingBid ~= 0: remainingBid <- bids[+j].size
    RETURN { size, gross, net, capital, bestEdge }

FEE(venue, p): RETURN 0.07 * p * (1 - p) IF venue == kalshi ELSE 0

COMPUTE-ARB(polyBook, kalshiBook, closeTime):
  d1 <- WALK(poly.asks, poly, kalshi.bids, kalshi) // buy YES poly, hedge kalshi
  d2 <- WALK(kalshi.asks, kalshi, poly.bids, poly) // buy YES kalshi, hedge poly
  best <- d1 IF d1.net >= d2.net ELSE d2
  IF best.net <= 0 OR best.size <= 0: RETURN EMPTY_ARB
  roi <- best.net / best.capital
  years <- (closeTime - now) / 365.25 days
  RETURN { roi, annualizedRoi: roi/years IF years > 0.02 ELSE null,
    avgEdge: best.net / best.size, ... }

BUILD-PAIRS(kalshiMarkets, polyMarkets, eventInputs):
  // --- Tier 0: event-level entity alignment -> VERIFIED
  FOR EACH (eventMatch, eventPolyMarkets) IN eventInputs:
    kInSeries <- kalshiMarkets WHERE seriesTicker == eventMatch.kalshiSeries
    kByEntity <- MAP canonicalEntity(m.subtitle, matchBy) -> m
    FOR EACH p IN eventPolyMarkets NOT IN usedP:
      k <- kByEntity[canonicalEntity(p.subtitle, matchBy)]
      IF not found AND matchBy == "team":
        k <- ARGMAX over unused kInSeries of
          |teamTokens(p.subtitle) INTERSECT teamTokens(m.subtitle)|, >= 1
      IF k found AND k NOT IN usedK: EMIT pair(p, k, VERIFIED)

  // --- Tier 1: curated seed -> VERIFIED
  FOR EACH s IN SEED_PAIRS not already consumed: EMIT pair(..., VERIFIED)

  // --- Tier 2: IDF-weighted fuzzy -> FUZZY
  remP, remK <- unconsumed markets ; df <- document frequency over remP + remK
  idf <- t -> log(1 + N / df[t])
  index <- inverted index: token -> [indices into remK] // blocking

  FOR EACH p IN the 600 most liquid remP, descending volume:
    candidates <- kalshi markets sharing >= 2 tokens, top 40 by shared-token count
    FOR EACH k IN candidates:
      IF p.category != k.category: CONTINUE // guard 1
      IF numbersConflict(p, k): CONTINUE // guard 2
      IF questionConflict(p, k): CONTINUE // guard 3
      IF isMultiEntity(p) OR isMultiEntity(k): CONTINUE // guard 4
      TRACK best WEIGHTED-SCORE(tokens(p), tokens(k)) >= 0.60
      IF best found AND pair.spread <= 0.35: EMIT pair(p, k, FUZZY)
      // else: title collision, not a match

  SORT BY (verified, fuzzy, unmatched), THEN spread DESCENDING

WEIGHTED-SCORE(a, b):
  shared <- a INTERSECT b
  IF |shared| < 2: RETURN 0
  IF NO t IN shared WITH df[t] <= 8: RETURN 0 // needs a distinctive token
  RETURN SUM(idf[t] for t in shared) / MIN(weight(a), weight(b))

```

Series resampling. Pick a per-range bucket width, cap the grid at 2,200 buckets, then forward-fill: for each bucket time g , emit the most recent PricePoint with $t \leq g$. Every series shares the identical grid, so the chart library's index-based x-layout is proportional to time and the two venues align exactly at every x .

F Event Registry and Divergence Assessments

Key	Label	Kalshi series	By	Field
pres-2028	2028 U.S. President	KXPRESPERSON	person	candidates
dem-nom-2028	2028 Dem. Nominee	KXPRESNOMD	person	candidates
rep-nom-2028	2028 Rep. Nominee	KXPRESNOMR	person	candidates
world-cup-2026	2026 World Cup	KXMENWORLD CUP	team	~46 countries
nba-2027	2027 NBA Champion	KXNBA	team	30 teams
nhl-2027	2027 Stanley Cup	KXNHL	team	32 teams

Table 7. The registry (lib/events.seed.ts). Adding a league or election is one entry: a **Polymarket** event slug plus a **Kalshi** series ticker. The sports entries alone contribute ~108 verified pairs.

Event	Status	Conf.	Basis
pres-2028	divergent	0.84	Kalshi resolves on <i>inauguration</i> (20 Jan 2029); Polymarket on the AP/Fox/NBC <i>race call</i> (~Nov 2028). Different source of truth, threshold and timing. Four divergence cases recorded, including death or withdrawal between call and inauguration.
rep-nom-2028	minor	0.78	Both resolve on winning <i>and accepting</i> the nomination. Polymarket adds a clause locking the result against a pre-election-day replacement; Kalshi is silent, though a completed-event reading agrees.
dem-nom-2028	minor	0.78	As above.
<i>fallback</i>	minor	0.50	Unjudged Politics pairs get a generic warning: venues commonly diverge on who decides, on candidate death or withdrawal, and on “wins” versus “inaugurated” timing.

Table 8. Resolution-equivalence assessments (lib/divergence.ts), produced by an LLM judge over each venue’s resolution text and reviewed. Baked in rather than evaluated per request.

G Build and Run

```
npm install
npm run dev          # http://localhost:3000
npm run build && npm run start  # production
```

No configuration required: **Kalshi** is live out of the box, and **Polymarket** works directly wherever DNS resolves it and over DoH where it does not. Expect ~17 s of cold corpus build at boot, absorbed by `instrumentation.ts` before the first request.

Verifying degradation. Point `POLY_GAMMA_URL` at an unroutable host (or set `POLY_DOH=off` on a DNS-blocking network): the badge should flip to `SNAPSHOT`, the banner of Figure 1 appear, **Polymarket** history render dashed, and every view stay functional.

Compiling this report. `tectonic report.tex`, or `pdflatex report.tex` run twice. Figures are in `report/figures/`.

H Interface Reference

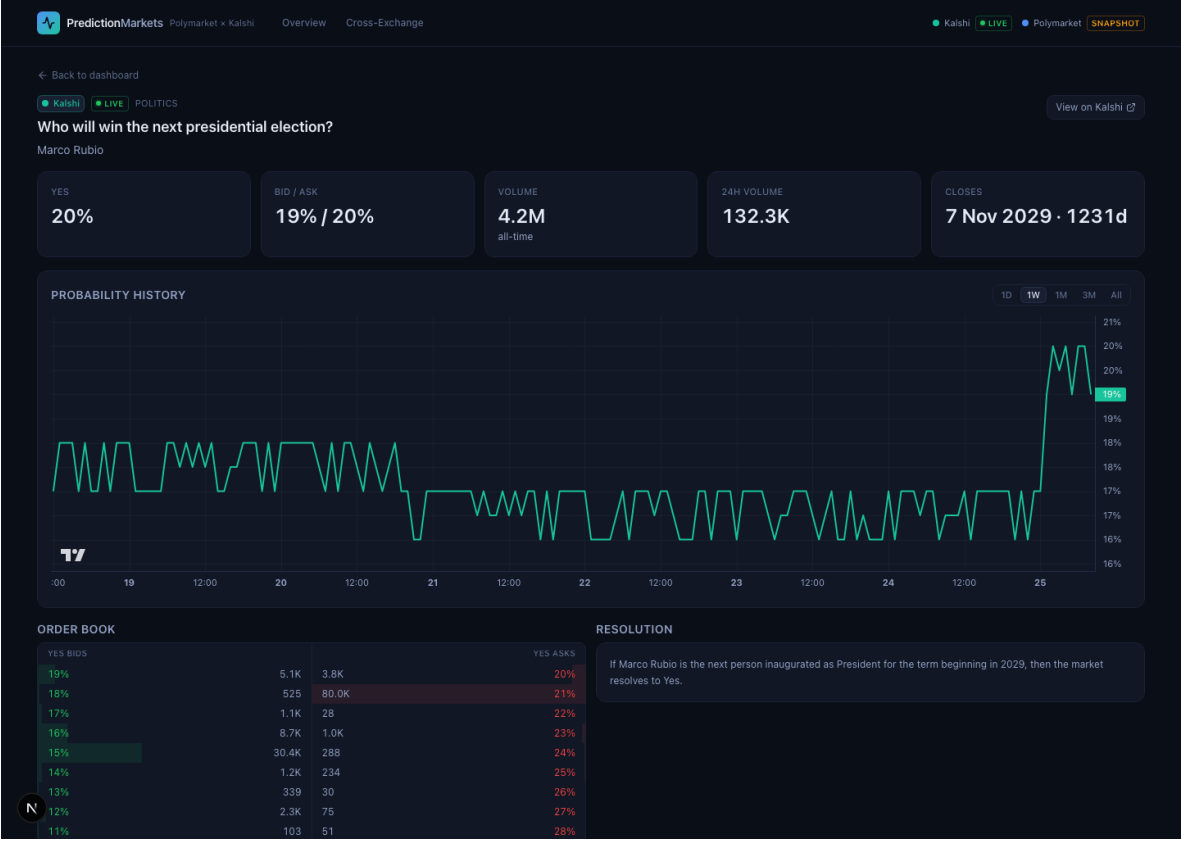


Figure 5. Single-market detail. Venue and data-mode badges, a deep link to the native venue, five statistic tiles, probability history with range selector, the full-depth book with its depth chart, and the venue’s complete resolution text — the same text the divergence judge consumes. Note the **Kalshi** wording here: “the next person inaugurated as President”.

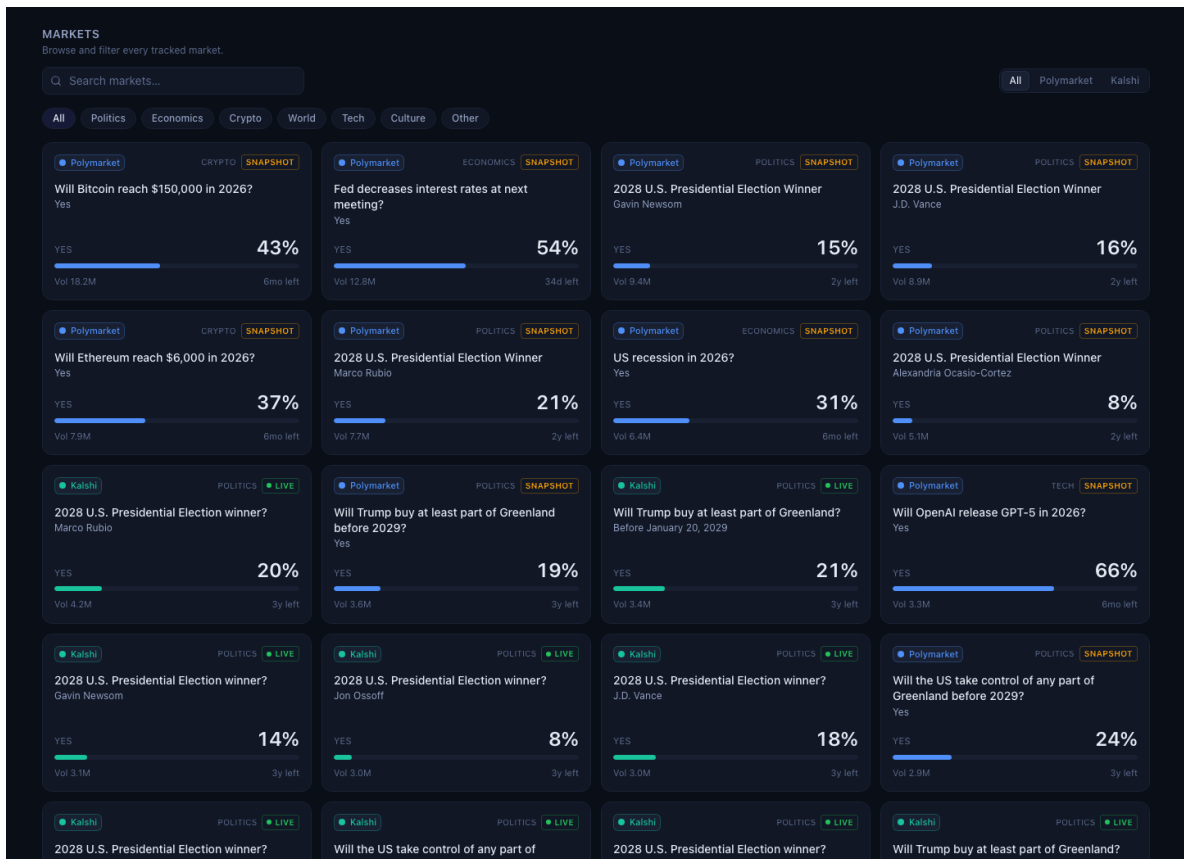


Figure 6. Market browser. The full normalised corpus across both venues with category pills, a venue segmented control, server-side sort, 300ms debounced search that fans out to both venues, and infinite scroll (IntersectionObserver, 700px root margin). Every card carries venue, category, data mode, price, volume in that venue’s own units, and time to close.